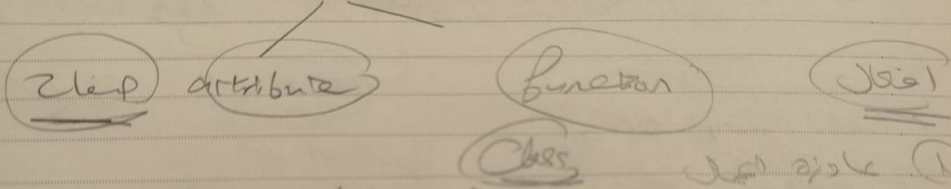


folders → files also can folder in package

Oop → object oriented Programming &

.Code // can be. Simple Code Style

Class can be used in 3 ways



src → click Right

new → Java Class

Class name → Capital Letter

Capital Letter and Capital Letter

Camel Case ~~name~~ functions & attributes

Student.java → Class end

Public Class Student {

Long id;
String name;
Float grade;

Attributes

Public void Print()

{ system.out.println("id" + id);

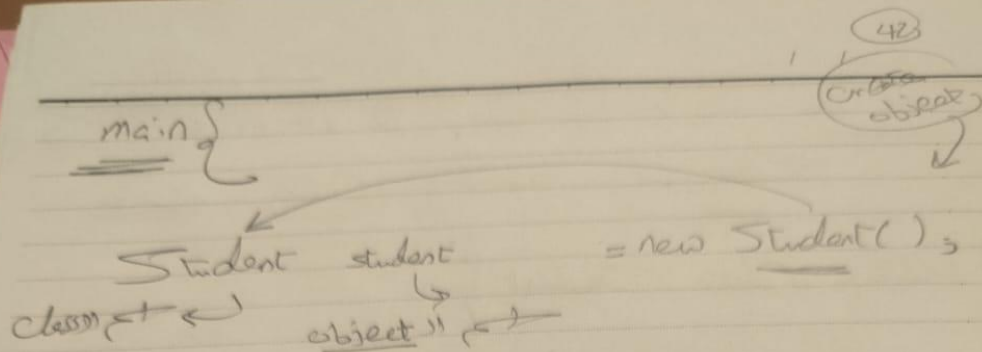
Prints

}

Main

// go to Class // edit class

Create object



Example Student class and Student object

Student class

Class and Pointer to object

Student id = 10;
Student name = "ahmed";
Student grade = 3.5;

System.out.println(student.id);

Student.print();

```
Public Class Student {
```

```
    void Print (int Count)
```

```
{    while (Count > 0)
```

```
{    System.out.println ("id" + id);
```

```
        Count--; }  
}
```

```
main {
```

```
    Student Print (2);
```

```
    Give id }  
}
```

Access modifier :-

Public class student {

Private Long id1 ;

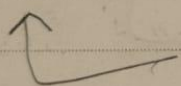
→ main() method Student class ko access karne ke liye use karne hai

Public Long id2 ; Class ko access karne ke liye use karne hai
package main ko access karne ke liye use karne hai

Protected Long id3 ;

Class ko access karne ke liye use karne hai
package main ko access karne ke liye use karne hai

Long id4 ;



default access modifier

default

Class ko access karne ke liye use karne hai
package main ko access karne ke liye use karne hai

Constructor -

class ki jaisi method

hota hai function ki jaisi

Public class Student {Student() {

"default Cons"

}

object jaisi hai agar object create karna hai toStudent (int Count) {while (Count > 0) { System.out.println("Student")
Count--;

"Parameter Cons"

Public class main {

Student student2 = new Student ();

default Cons. ای میخواند

default Constructor ای میخواند

Student student2 = new Student (count 5);

Parameter Cons. ای میخواند

و این (5) Student میخواند

ای default Constructor ای میخواند

ای Parameter Cons. ای میخواند

ای default ای میخواند

over loading →
etc. Datatype " " ...
Parameters " "

Boolean Connection DB = false;

Public void PrintFromDB() {
if (ConnectionDB)

{ System.out.println("Go to db..."); }

else { System.out.println("no Connection found"); }

return false; }
no Connection found

true ? Print (s);

Public class main {

Student student = new Student();

student.ConnectionFromDB;

Public class Student {

Private Boolean ConnectionDB = false;

Public void PrintFromDB() {

if (ConnectionDB) {

System.out.println("go to db");

else { System.out.println("no Connection found"); }

Public Boolean OpenConnection(String username,
String Password) {

if (username == "admin" && Password == "admin123")

{ ConnectionDB = true; }

return ConnectionDB;

}

Public class main {

49

Student student = new Student ();

Boolean ConnectionOpen = student.openConnection(
its variable ←
username: "ahmed", Password: "123");

if (ConnectionOpen) { student.Print From DB(); }

else { System.out.println ("invalid username
or Password"); }

(invalid username or
Password)

yes ←

(username: "admin", Password "admin123") yes

(go to db)

yes ←

Constructor // Password 123456789

Public class Student {

Private Boolean Connection DB = False;

Public Student(String username, String Password)

{ OpenConnection (username, Password); }

Code // 20.10.2020

Public void PrintFromDB()

return Connection DB;

Public main Class {

Student Student1 = new Student (username: "admin",
Password: "admin123");

Student1. PrintFromDB();

Polymorphism

encapsulation:-

```
Public class Student {
```

```
    Private Long id;
```

```
    Private String userName;
```

```
    Private String Password;
```

```
    Public Long getId () { return id; }
```

```
    Public String getUserName () { return userName; }
```

```
    Public String getPassword () { return Password; }
```

```
    Public void setId (Long value) {
```

```
        if (value > 0) { id = value; }
```

```
        else { System.out.println("invalid id"); } }
```

```
    Public void setUsername (String value) {
```

```
        if (value.length() >= 8) { userName = value; }
```

```
        else { System.out.println("invalid userName"); } }
```

```
    Public void setPassword (String value) {
```

```
        if (value.length() >= 8) { Password = value; }
```

```
        else { System.out.println("invalid Password"); } }
```

```
    Public void PrintData () {
```

```
        (id == Null && userName == Null && Password == Null)
```

(52)
{ system.out.println("no data found");
return; }

system.out.println("----- Student Data -----");

if (id != null)
{ system.out.println("id : " + id); }

if (userName != null)
{ system.out.println("userName : " + userName); }

if (password != null)
{ system.out.println("Password : " + password); }

public class main {

Scanner scanner = new Scanner(System.in);

system.out.println("welcome student");

system.out.println("Pls enter your id");

Long id = scanner.nextLong();

system.out.println("Pls enter your username");

String name = scanner.next();

system.out.println("Pls enter your Password");

String password = scanner.next();

(53)

11) Student.PrintData();

{ id = value ; }

① ریاستی لکھ

②

$$\{ \underline{\text{this.id}} = \text{id}_3 \}$$

الكوافصة الى Class عاقل لا يدرك

Inheritance

①

Base Entity

الأساسي

52853 PUCAL مع main

فصل 11

Base Entity

class

Student & Teacher

الطلاب

Public Class BaseEntity {

Private long id;

Private String userName;

Private String password;

Public long getId() { return id; }

Public String getUserName() { return userName; }

Public String getPassword() { return password; }

Public void setId(long value) {

if (value > 0) { id = value; }

else { System.out.println("invalid id"); } }

Public void setPassword(String value) {

if (value.length() >= 8) { password = value; }

else { System.out.println("invalid password"); } }

(55)

```
Public void SetPassword (String value) {  
    if (value.length() >= 8) { Password = value; }  
    else { system.out.println ("invalid Password"); } }  
}
```

```
Public Boolean PrintBaseData () {
```

```
    if (id == null || userName == null || Password == null)  
    { system.out.println ("no data found");  
      return false; }
```

```
    system.out.println ("----- Student data -----");
```

```
    if (id != null) { system.out.println ("id: " + id); }  
    if (userName != null) { system.out.println ("userName: " + userName); }  
    if (Password != null) { system.out.println ("Password: " + Password); }
```

```
    return true; }
```

```
Public class Student extends BaseEntity {
```

```
    Private Float grade;
```

```
    Public Float getGrade ()  
    { return grade; }
```

```
    Public void SetGrade ( Float grade)  
    { this.grade = grade; }
```

1 1 56
public void PrintData() {
if (!PrintBaseData() && grade == null)
{ System.out.println("no data found");
return; }

System.out.println("----- Student data -----");

if (grade != null)

{ System.out.println("grade: " + grade); }

public class Teacher extends BaseEntity {

private Float salary;

public Float getSalary() { return salary; }

public void setSalary (Float salary)
{ this.salary = salary; }

public void PrintData()

{ if (!PrintBaseData() && salary == null)

{ System.out.println("no data found");

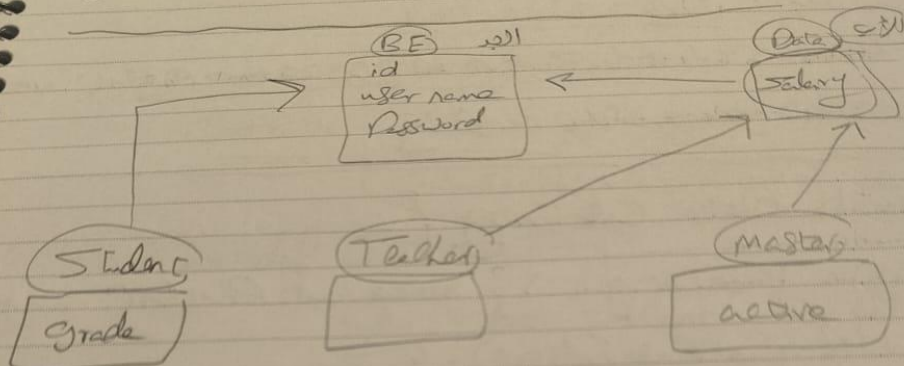
return; }

1 1 (53)

```

system.out.println("----- Teacher data -----");
if (Salary != null)
{ system.out.println("Salary : " + Salary); }

```



Classes // Using mult: inheritance

class BaseEntry

class Data extends

BaseEntry

class Student extends BaseEntry

(Teacher & Master) extends Data

(Data)

Inheritance [Constructor]

1 (58)

```
Public class Student extends BaseEntity {
```

```
    Public Student () { System.out.println("i am on  
    def Con"); }
```

```
    Public Student (String name) { System.out.println("i am  
    Student myName = " + name); }
```

```
Public class BaseEntity {
```

```
    Public BaseEntity () { System.out.println(" i am  
    on def Con BaseEntity"); }
```

```
Public class main {
```

15.9.2020

```
    Student student1 = new Student ();
```

```
    System.out.println("-----");
```

```
    Student student2 = new Student (name: "ahmed");
```

class Student > BaseEntity

BaseEntity

baseEntity > object Entity > >

... > > >

Entity object > > >

Constructor > > > Student

Student > object > >

iam on def Can BaseEntity

class Student > >

iam on def Can

Entity > > object > >

iam on def Can BaseEntity

iam Student my name : ahmed

1. Constructors in Java Class used
C++ Constructor - no default constructor
all default 1) 1) default constructor

```
Public class Student extends BaseEntity {
```

```
Public Student() {
```

```
Super(value: "default");  
System.out.println("i am on def con");
```

```
Public Student(String name) {
```

```
Super(name);
```

```
{ System.out.println("i am Student my name: " + name); }
```

```
Public Student(Integer id, String name) {
```

```
Super(id, name);
```

```
{ System.out.println("i am Student my id: " + id  
+ "name: " + name); }
```

```
Public class BaseEntity {
```

```
Public BaseEntity(String value) {
```

```
System.out.println("i am on con BaseEntity value: " + value); }
```

```
Public BaseEntity(Integer value1, String value2) { System.out.println  
(i am on con BaseEntity value: " + value1 + "value2: " + value2); }
```

Public class main {

Student student1 = new Student();

System.out.println("-----");

Student student2 = new Student(name: "ahmed");

System.out.println("-----");

Student student3 = new Student(id: 1,
name: "ahmed");

نوع لايه Super " " " " " " " " " " " "

Parameter Constructors
Student " " " " " " " " " " " "

... Student constructor
iam on Con BaseEntity value: default
iam on def Con

iam on Con BaseEntity value: ahmed
iam Student my name: ahmed

iam on Con BaseEntity value1: 1 value2: ahmed
iam Student my id: 1 name: ahmed

1 (12)
output of one class default constructor used in other class
Super constructor

```
public Student() {  
    System.out.println("i am on def con");  
}
```

```
Super(value: "default"); // (X) error  
// it is not Super constructor
```

```
public class Teacher extends Data {  
    public Teacher() {  
        Super(value: "ahmed");  
        System.out.println("i am on def con Teacher");  
    }  
}
```

```
public Teacher(String value) {  
    System.out.println("i am on con Teacher value: " + value);  
}
```

```
public Teacher(Integer value1, String value2) {  
    System.out.println("i am on con Teacher value1: " + value1 + "value2: " + value2);  
}
```

```
public class Data extends BaseEntity {  
    public Data() {  
        System.out.println("i am on def con Data");  
    }  
}
```

```
System.out.println("i am on def con Data");  
}
```

```
public Data(String value) {  
    System.out.println("i am on con Data value: " + value);  
}
```

```
public Data(Integer value1, String value2) {  
    System.out.println("i am on con Data value1: " + value1  
        + "value2: " + value2);  
}
```

```

public class BaseEntity {
    public BaseEntity() {
        System.out.println("i am on def. Can BaseEntity");
    }
    public BaseEntity(String value) {
        System.out.println("i am on Can BaseEntity value: " + value);
    }
    public BaseEntity(Integer value1, String value2) {
        System.out.println("i am on Can BaseEntity values: " + value1 +
            "value2: " + value2);
    }
}

```

```

public class main {

```

```

    Teacher teacher = new Teacher();

```

default constructor Teacher → 200 Jol 2000
 constructor → 200 Jol 2000 → 2000 case
 constructor → 200 Jol 2000 → 2000 case
 constructor → 200 Jol 2000 → 2000 case
 constructor → 200 Jol 2000 → 2000 case

```

    iam on def. Can BaseEntity
    iam on Can Data values ahmed
    iam on def. Can teacher

```

Comment: BaseEntity is default Can 2000 case
 Data class → 2000 case
 Error

Protected & default are class
in inheritance default is not
inherit no Java Package

بالنسبة الى Protected في inherit هي بين شيئين
شيء من Package لا شيء من Package
بدا

Public class BaseEntity {

int id1; → من Package 1 و من Class 1
Protected int id2; →

Public class Student extends BaseEntity {

Public void Print () {

System.out.println("id1" + id1);

↳ Access Package 1 و من Package 1
Error عند Package 1

System.out.println("id2" + id2); }

Package 1 ✓
Package 1 ✓

(15)

(Inheritance) (Override) (Copy)

```

    public class Student extends BaseEntity {
    public void Print() {
        System.out.println("my class is Student");
    }
    }

    public class BaseEntity {
    public void Print() {
        System.out.println("my class is BaseEntity");
    }
    public void Print(String name) {
        System.out.println("my class is BaseEntity");
        System.out.println("my name is: " + name);
    }
    public void Print(Integer id, String name) {
        System.out.println("my class is BaseEntity");
        System.out.println("my id is: " + id);
        System.out.println("my name is: " + name);
    }
    }
  
```

```

    public class main {
  
```

```

        Student student = new Student();
        student.Print();
  
```

```

        student.Print("Islam");
  
```

```

        student.Print(id=1, name="Islam");
  
```

K. A. J.

1 1 (16)
 override class ke print() function
 class ke print() function
 Inheritance is allording
 life cycle
 implementation
 body

والتا يقول متشوقش الالاب والاب وقتنا
 الالاب والاب والاب

my class is Student

my class is BaseEntity
 my name is Islem

my class is BaseEntity
 my id is: 1
 my name is: islem

Super الالاب الالاب الالاب الالاب
 Public class Student extends BaseEntity

```
Public void Print() {
    super.Print();
    System.out.println("my class is Student");
}
```

الالاب الالاب الالاب الالاب

my class is BaseEntity
 my class is Student

clean code

→ Override في الـ code

67
الـ code

Public class Student extends BaseEntity {

Ca override

Public void Print() {

Ca override → header الـ code في الـ code
override في Compiler الـ code

الـ code الـ code الـ code الـ code
override الـ code الـ code الـ code

Final

Public class BaseEntity {

Public final void Print (String name) {

System.out.println("my name is" + name);

الـ code الـ code الـ code
Print الـ code

Public class Student extends BaseEntity {

Ca override

Public void Print (String name) {

}

(Final) → class → class
 main → object → class → class
 @main → class → class
 Public final class BaseEntity { }
 Public class main {
 BaseEntity baseEntity = new BaseEntity();
 Public class Student extends BaseEntity { }
 Final class → error
 Object → class
 (X) → class

abstract class
 abstract class
 abstract class

Public abstract class BaseEntity {
 Public void Print() {
 System.out.println("my class is BaseEntity");
 }

در واقع این یک روش برای چاپ و نمایش است
 override

1. 1. (15)
 public final void Print (String name) {
 }
 }
 child n joled as
 final class @ override

public abstract void Print (Integer id, String name);

child n joled as
 header n joled as
 abstract class @ override

(2P)

final

joled as class

(X) object n joled

(✓) joled

abstract

class

final class

abstract class

object

✓

✓

X

Inherit

✓

X

✓

function → override joled

final function → child n joled as

abstract function → child n joled as

70
override // Java Polymorphism //

1. Method Overriding - It is a way of overriding the method of the superclass with the same name and signature in the subclass. It is used to provide a specific implementation of the method that is already defined in the superclass.

Public class BaseEntity {

Public void Print () {
System.out.println("my class is BaseEntity");}

Public class Teacher extends BaseEntity {

Override

Public void Print () {
System.out.println("my class is Teacher");}

Public class Master extends BaseEntity {

Public void Print () {
System.out.println("my class is Master");}

Public class main {

Student student = new Student ();
student.Print ();

Teacher teacher = new Teacher ();
teacher.Print ();

Master master = new Master ();
master.Print ();

1 / 1 (73)
class abstract class
object لا يمكن ان يكون
default constructor
لا يمكن ان يكون

```
public abstract class BaseEntity {
```

```
public BaseEntity() {
```

```
System.out.println("hi BaseEntity");
```

```
public abstract void PrintAllData();
```

```
public class Student extends BaseEntity {
```

```
public Student() {
```

```
System.out.println("hi Student");
```

Override

```
public void PrintAllData() {
```

```
public class main {
```

```
Student student = new Student();
```

hi BaseEntity
hi Student

20/5/20

(72)

default class contain (default func. → final func.)
 abstract class contain (default func. → final & abstract)
 abstract class can't create object but can extend
 final class can create object but can't extend
 default class, abstract class → extends or class
 interface → implements

Public interface BaseEntity {

void Print1();

void Print2();

void Print3();

Public class Student implements BaseEntity {

Override

Public void Print1() {

System.out.println("Student Print1"); }

Override

Public void Print2() {

System.out.println("Student Print2"); }

Override

Public void Print3() {

System.out.println("Student Print3"); }

Public class Teacher implements BaseEntity {

Override

Public void Print1() {

System.out.println("Teacher Print1");

Override

Public void Print2() {

System.out.println("Teacher Print2");

Override

Public void Print3() {

System.out.println("Teacher Print3"); }

Public class main {

BaseEntity baseEntity = new Student();

baseEntity.Print1();

baseEntity.Print2();

baseEntity.Print3();

object class use

baseEntity as

Student object

baseEntity = new Teacher();

baseEntity.Print1();

baseEntity.Print2();

baseEntity.Print3();

object of class use

class as

Teacher object

Polymerism

It's like

Student Print1
Student Print2
Student Print3

Teacher Print1
Teacher Print2
Teacher Print3

Student student = new Student();
Student is to show all functions // ki dehi nahi

Base Entry baseEntry = new Student();

override

show all functions // ki dehi nahi
// ki dehi nahi

Public class employee {

Private Integer Counter = 0;

Public void createEmployee (String name) {
System.out.println("hi employees" + name);
Counter++;
}

Public void showEmployee () {
System.out.println("Employee Count: " + Counter);
}

Public Class main {

```
Employee employee1 = new Employee();  
employee1.CreateEmployee(name: "ahmed");  
employee1.ShowEmployeeCount();
```

memory v.p collection and object

```
Employee employee2 = new Employee();  
employee2.CreateEmployee(name: "ahmed");  
employee2.ShowEmployeeCount();
```

some of the memory is in the collection and object

شخصي

hi employee: ahmed
employee Count: 1

hi employee: ahmed
employee Count: 1

Class Counter

some of the memory is in the collection and object

Static is an object

Private Static Integer Counter = 0;

Static Class ١١ Counter ١٢

inlog class ١٣
 1) write to the object ١٤
 hi employee : ahmed
 Employee Count : 1
 hi employee : ahmed
 Employee Count : 2

Static → Shared Attribute ١

Static Class * Ball ca need ٢

System.out.println("Employee: Counter");

inlog class ٢ ١٥

Public class ١٦

Public Static Integer Counter = 0;

Static Function ١٧

Public Static void CreateEmployee (String name) {
 System.out.println("hi employee: " + name);
 Counter++;
}

لقد انشأنا دالة Public لا
 داخل class و ليس داخل object

Public class main {

Employee createEmployee(name: "ahmed");

Employee e = ahmed

System.out.println(e.name);

Public interface SchoolService {

void add();

void remove(); }

Public interface StudentService {

void addStudent();

void removeStudent(); }

Public interface TeacherService {

void addTeacher();

void removeTeacher(); }

Interface cannot implement -> class class

✓ or not

(78)

```
Public Class StudentServiceImpl implements School Services  
StudentService, TeacherService {
```

```
    @Override  
    Public void add() {  
        System.out.println("add");  
    }
```

```
    @Override  
    Public void remove() {  
        System.out.println("remove");  
    }
```

```
    @Override  
    Public void addStudent() {  
        System.out.println("addStudent");  
    }
```

```
    @Override  
    Public void removeStudent() {  
        System.out.println("removeStudent");  
    }
```

```
    @Override  
    Public void addTeacher() {  
        System.out.println("addTeacher");  
    }
```

```
    @Override  
    Public void removeTeacher() {  
        System.out.println("removeTeacher");  
    }
```

Public class main {

SchoolServiceImpl schoolService = new SchoolServiceImpl();

schoolService.add();

schoolService.remove();

schoolService.addStudent();

schoolService.removeStudent();

schoolService.addTeacher();

schoolService.removeTeacher();

Interface is an abstract class

StudentService schoolService
= new SchoolServiceImpl();

function is used to
call Student up to school
(array) and return

add
remove
add Student
remove Student
add teacher
remove teacher

implement is used to interface
is an interface

interface

interface is class which is not implement

